Information Sciences 675 (2024) 120743



Contents lists available at ScienceDirect

Information Sciences

journal homepage: www.elsevier.com/locate/ins





HUSM: High utility subgraph mining in single graph databases

Zhaoming Chen [a], Cheng He [a], Guoting Chen [b,*], Wensheng Gan [c], Philippe Fournier-Viger [d]

[a] School of Science, Harbin Institute of Technology, Shenzhen, China
[b] School of Science, Great Bay University, Dongguan, Guangdong, China
[c] College of Cyber Security, Jinan University, Guangzhou, China
[d] College of Computer Science and Software Engineering, Shenzhen University, China

ARTICLE INFO

Keywords:
Utility pattern mining
Concise representation
Graph mining
Pattern mining

ABSTRACT

Frequent subgraph mining (FSM) is a crucial research area with diverse applications. However, traditional FSM treats all subgraphs as equally important. In practical applications, some subgraphs may be frequent but not very valuable, while others may be infrequent but highly valuable. To address this issue, we introduce utility pattern mining into single graph mining, enabling the discovery of meaningful patterns based on user's interest rather than relying solely on support. We define a utility function in single graph databases and formally define the problem of high utility subgraph mining (HUSM). In the face of several challenges posed by this new problem, such as the absence of *downward closure property* and a large number of candidates, we design several utility upper bounds that satisfy the *downward closure property*. We then develop a HUSM algorithm to efficiently mine all high utility subgraphs in single graph databases. Additionally, we design a lossless concise representation for high utility subgraphs, which has fewer instances than the total number of high utility subgraphs and can derive all of them, making the mining results more representative. The experimental results demonstrate that our method exhibits excellent performance, and the concise representation plays a significant role in simplifying the mining results.

1. Introduction

Frequent pattern mining (FPM) is a fundamental task in data mining, aiming to discover patterns in a given database that occur with a frequency not lower than a user-specified threshold. Over the past decades, this support-based research has been extensively explored for various types of databases, such as transaction databases [2,16], event sequences [15,41,44], and graph databases [19,24,40,42], to discover frequently occurring substructures such as itemsets, sequences, episodes, and subgraphs. Graph models have become increasingly crucial for representing complex structures, such as chemical molecular structures, social networks, and the Web. Traditional frequent subgraph mining involves counting the individual graphs containing a subgraph in a given multi-graph database [9,18] to compute the subgraph's support and then determine whether it is frequent. In recent years, a new wave of frequent pattern mining for single graph databases [1,12,37] has emerged, and these problems are often even more challenging.

* Corresponding author.
E-mail addresses: 22s058004@stu.hit.edu.cn (Z. Chen), 22s058015@stu.hit.edu.cn (C. He), guoting.chen@univ-lille.fr (G. Chen), wsgan001@gmail.com (W. Gan), philfv@szu.edu.cn (P. Fournier-Viger).

https://doi.org/10.1016/j.ins.2024.120743
Received 20 December 2023; Received in revised form 5 March 2024; Accepted 12 May 2024

Available online 16 May 2024
0020-0255/© 2024 Elsevier Inc. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

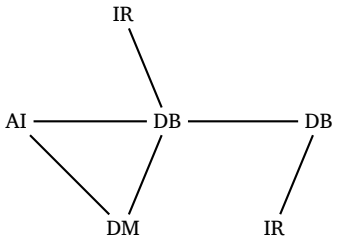


Fig. 1. Collaboration graph.

However, FPM solely relies on support as an interestingness measure, treating a pattern as more interesting if it appears more frequently. However, for specific scenarios, this metric may not be suitable. For example, in frequent itemset mining, consider a shopping transaction database with {*milk, bread*} occurring ten times and having a unit value of 5, resulting in a total value of 50, and {*red wine*} appearing twice but with a unit value of 100, yielding a total value of 200. In this scenario, {*red wine*} generates more values despite its lower frequency. Therefore, traditional FPM falls short of capturing valuable information.

A mining framework inspired by utility theory, known as utility pattern mining (UPM) [3,4,23,39], has been introduced to address this challenge. The utility of a pattern can be measured in terms of profit, external utility, cost, or other metrics depending on user preferences. For example, as mentioned earlier, the utility of {*red wine*} can be calculated as the product of its unit price and the quantity purchased. UPM aims to discover patterns in a given database that have a utility not lower than a user-specified threshold, denoted as *minUtil* (minimum utility). UPM has been extended in diverse research in data mining in recent years, including high utility itemset mining, high utility sequential pattern mining, and high utility subgraph mining in graph transactions. Several methods integrate the external utilities of subgraphs in single graph databases. These algorithms incorporate weight information in graph data and define various weighting functions to perform mining tasks from different perspectives. For example, WIGM [43] introduces a 1-extension property to accomplish mining tasks, RESUM [36] utilizes a series of *MNI*-compatible scoring functions to measure the value of subgraphs, and WeFreS [5] introduces a *MaxPosW* strategy to mine subgraphs considering *MNI* support and edge weights, effectively pruning the search space to complete mining tasks. WeGraMi [29] considers node weights in graph data and uses a *MaxMin* measure to quantify the weight of subgraphs, satisfying the anti-monotonicity property. OWGraMi [27] optimizes WeGraMi by removing infrequent edges and performing weight calculations based on parent subgraph information. AWeGraMi [28] improves the weight calculation of WeGraMi, adopting an *AveMin* strategy to consider the average value in calculating subgraph weights, and successfully performs mining tasks.

We apply the utility-based mining methods to design a different function considering the internal utilities, such as the occurrence frequency of subgraphs, and the external utility, such as the value of vertex labels. And this framework can design the mining method from another perspective.

Fig. 1 shows a collaboration graph in which each vertex represents a researcher, and each vertex's label denotes their research field, with AI indicating artificial intelligence, DB standing for databases, and DM for data mining. Each edge connecting two vertices indicates research collaboration between the corresponding researchers. In this example, users may be more interested in structures that include DM, but frequent subgraph mining typically relies on occurrence frequency as a metric. Therefore, subgraphs containing DM may not be frequent since only one vertex labeled DM in Fig. 1. Utility-based subgraph mining can assign higher external utility to the label DM while using the frequency of a subgraph (its number of occurrences) as an internal utility. The value of a subgraph can consider its internal and external utilities. This approach enables the discovery of structures that are more appealing to users.

However, the *downward closure property* commonly used for pruning the search space in FPM does not hold in UPM. Consequently, UPM is not a straightforward task. To conduct high utility pattern mining, many studies incorporate the concept of *transaction weighted utilization* (*TWU*) [14,31,45]. *TWU* serves as a utility upper bound and is frequently employed as a fundamental pruning technique to reduce the search space. Pruning with *TWU* ensures the satisfaction of the *downward closure property*. For a pattern to qualify as a candidate, its *TWU* must satisfy the user-specified *minUtil* threshold. Following this, we calculate the utility for all candidates and get the high utility patterns from them. Similarly, in existing methods for high utility subgraph mining in graph transaction databases, analogous utility upper bounds, such as the *graph weighted utility* (*GWU*) [4,23], have been proposed and utilized to prune the search space.

However, *GWU* cannot be directly applied to single graph databases. There are the following challenges when dealing with the problem of mining high utility subgraphs in single graph databases.

(i) The utility function for single graphs lacks the *downward closure property*, and there is no explicit relationship between the utility of a subgraph and its supergraph. Existing frequent subgraph mining often relies on the support's *downward closure property*, which cannot be directly applied here.

(ii) Mining subgraphs in a single graph database is a complex task. We must ensure that the mined graphs are indeed subgraphs of the data graph, which involves the subgraph isomorphism problem – an NP-complete problem. Additionally, the number of occurrences of each subgraph in the data graph does not satisfy the *downward closure property*, either.

(iii) Existing UPM mining methods cannot be applied in single graph databases since they are designed for transaction databases. This means that utility upper bounds similar to *GWU* do not apply to single graph databases, and we need to redesign a fundamental utility upper bound.

(iv) After designing a fundamental utility upper bound, we must investigate efficient methods to reduce the number of candidates, enabling efficient execution of the high utility subgraph mining task.

In addition, the final subgraphs generated may have many information overlaps. Therefore, in frequent pattern mining, users search for special types of patterns that are representative and can derive all other frequent patterns. These patterns are referred to as concise representations of frequent patterns. The two most representative concise representations are maximal and closed frequent patterns [6,33]. In the context of frequent subgraph mining, these are referred to as maximal frequent subgraphs and closed frequent subgraphs [8,22]. Maximal frequent subgraphs are those for which none of their supergraphs are frequent, while closed frequent subgraphs are those for which their immediate supergraphs do not have the same support. These two subgraphs provide an effective concise representation of frequent subgraphs, and their number is much smaller than that of all frequent subgraphs. However, the effectiveness of these two concise representations relies on the *downward closure property* of support, which cannot be applied in high utility subgraph mining. Therefore, designing an effective concise representation for high utility subgraph mining is necessary.

This paper proposes an algorithm HUSM (high utility subgraph mining) for mining high utility subgraphs in single graph databases to address the above challenges. Our key contributions are as follows.

- We incorporate the concept of utility into graph mining in single graph databases and formalize the problem of high utility subgraph mining. We also design an exact and efficient algorithm HUSM, to mine all high utility subgraphs.
- We design three utility upper bounds called *initial vertex utility* (*IVU*), *extension space utility* (*EU*) and *CMNI extension space utility* (*CEU*). These upper bounds satisfy the *downward closure property*, allowing for effective high utility subgraph mining tasks. Among them, the tighter upper bound *CEU* can effectively reduce the number of candidates, enabling the HUSM algorithm to perform mining tasks efficiently.
- We design a lossless concise representation for high utility subgraphs utilizing the DFS code. This concise representation is referred to as *boundary high utility subgraph* (*BHUS*). The number of *BHUS* is smaller than the total number of all the high utility subgraphs, and it can derive all high utility subgraphs. Additionally, we provide methods for obtaining *BHUS* and deriving all high utility subgraphs from *BHUS*.
- We collect six real single graph datasets and conduct experiments. The experimental results demonstrate that HUSM can mine all high utility subgraphs. The utility upper bound *CEU* ensures mining efficiency, and *BHUS* serves as an effective concise representation.

The rest of the paper is organized as follows. Section 2 introduces related studies. Section 3 presents the necessary preliminary concepts. Section 4 describes our high utility subgraph mining method for single graph databases. Section 5 discusses the design of high utility concise representations. Section 6 presents experimental results on six real datasets. Section 7 concludes the paper and outlines directions for future work.

2. Related work

We present related work in three aspects: frequent subgraph mining, weighted subgraph mining, and utility pattern mining.

2.1. Frequent subgraph mining

Discovering frequent subgraphs in graph transaction databases is a classic task in frequent subgraph mining (FSM). There are many methods including AGM [19], FSG [24], FFSM [18], and gSpan [42]. The support measures of these algorithms all satisfy the *downward closure property*. Among these algorithms, gSpan is widely used for various graph pattern mining tasks. It constructs a minimum DFS Code tree, where each node represents a graph, and it generates new graphs by adding an edge to the vertices on the rightmost path.

In the realm of FSM for single graph databases, the conventional approach using the number of isomorphisms as the support metric falls short of satisfying the *downward closure property* or anti-monotonicity property. Consequently, researchers have ventured into alternative support measures, such as the *maximum independent set based support (MIS)*, an anti-monotonic support measure hinging on overlapping graphs. However, the associated mining task is established as NP-hard [26,40]. Another measure, *harmful overlap (HO)*, assesses the value of vertex overlaps, but the problem is also NP-hard [7]. In contrast, *minimum image based support (MNI)* [13] leverages vertex images, requires only linear time for computation, and aligns with the *downward closure property*. Several algorithms have emerged to make use of these support measures. SIGRAM [26] relies on *MIS* to identify frequent subgraphs in large sparse graphs, demanding the reconstruction of all computational steps. GREW [25] adopts a heuristic approach for extension space pruning, but it cannot find all frequent subgraphs. GraMi [12] only retains templates of the frequent subgraphs and does not store the embeddings. It employs *MNI* as its support measure and conducts frequency evaluation of constructed subgraphs by modeling constraint satisfaction problem (CSP). And it proposes a series of optimizations to enhance its performance. Furthermore, some algorithms optimize the process of frequent subgraph mining based on GraMi. ScaleMine [1] introduces a scalable parallel system for executing the process of frequent subgraph mining. The algorithm is two-phase, with the first phase being approximate, used to gather information about frequent subgraphs with high probability, and the second phase utilizing the information to complete the mining task. SSIGRAM [37] is based on Spark to parallelly execute the subgraph extension and the support counting. It also proposes heuristic search strategies and three optimizations to improve the efficiency of support calculation. Reference [35] presents three optimization strategies for enhancing GraMi: SuGraMi preserves the support information of frequent subgraphs based on GraMi,

SoGraMi sorts frequent edges by their supports and extends subgraphs utilizing the ordered edges to balance the search space and reduce candidates, PaGraMi adopts a parallel strategy, establishing a thread for each frequent edge and running them simultaneously to reduce the algorithm's runtime. In summary, GraMi is an efficient mining framework widely utilized in the field of single graph mining, and our algorithm is designed based on the GraMi framework.

2.2. Weighted subgraph mining

Another type of subgraph mining deals with weighted graph databases. This can be regarded as incorporating weight information into the mining process.

Jiang et al. [21] designed three edge-weighting strategies: average total weighting (ATW), affinity weighting (AW), and utility based weighting (UBW), and combined them with gSpan for mining frequent subgraphs in longitudinal social network data. Eichinger et al. [11] demonstrated that mining subgraphs is more accurate by including weights and proposed an approximate mining algorithm. MWSA [30] utilizes two metrics, support affinity and weight affinity, which satisfy the anti-monotonicity property, to mine subgraphs. WFSM-MaxPWS [20] employs a modified canonical representation of subgraphs to better align with its weight function and proposes a technique called maximum possible weighted support (MaxPWS) for pruning the search space. These algorithms are all targeted at weighted graph transaction databases.

WIGM [43] focuses on mining large weighted graphs and introduces the 1-extension property to address the problem that the weighting function does not satisfy the anti-monotonicity property. In the same study, a top-k version of the algorithm is also presented. RESUM [36] utilizes a series of *MNI*-compatible scoring functions to handle graphs with multiple edge weights, and both approximate and distributed versions of the algorithm are provided. WeFreS [5] considers the *MNI* support and edge weights, defining weighted support to measure the value of subgraphs. It is used to mine all subgraphs in the graph data with weighted support not less than a given threshold. As weighted support does not satisfy the *downward closure property*, WeFreS introduces a *MaxPosW* measure as an upper bound for the weighted support of a subgraph and its descendants. The algorithm effectively prunes the search space by applying this metric and a series of related operations. All these algorithms deal with single large edge-weighted graphs. WeGraMi [29] takes into account vertex weights in graph data and employs a *MaxMin* measure for weight calculation of the subgraphs. The weight of a subgraph defined by this strategy satisfies the anti-monotonicity property. The algorithm applies the *MaxMin* measure and *MNI* support as two metrics to mine frequent weighted subgraphs in graph data. OWGraMi [27] is an optimization of WeGraMi, mainly involving two strategies. The first strategy prunes all infrequent edges before mining to reduce the search space. The second strategy is to compute the weight of a subgraph based on the weight information of its parent subgraph, and this strategy can improve the efficiency of subgraph weight calculation. AWeGraMi [28] enhances WeGraMi from the perspective of weight calculation, considering the average value in the domain during weight calculation of a subgraph, and this process is called *AveMin* measure. The algorithm applies the *MaxMin* measure to calculate the weight of subgraphs, serving as an upper bound for *AveMin* and effectively pruning the search space. Our algorithm proposes a different function that considers subgraph support as internal utility and the value of the vertex labels as external utility. Moreover, we leverage the utility mining framework to design our algorithm.

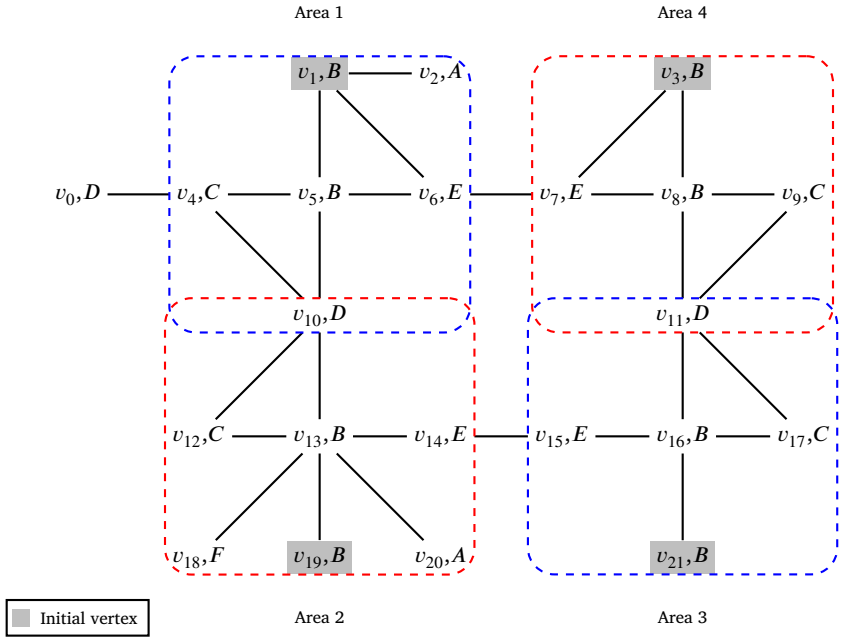
2.3. Utility pattern mining

Among the various utility pattern mining (UPM) methods, high utility itemset mining (UIM) is the most extensively researched and explored. Early complete UIM methods typically follow a two-phase approach, where all candidate itemsets are obtained in the initial phase, followed by identifying high utility itemsets from these candidates in the second phase. A representative algorithm, Two-Phase [32], draws inspiration from the Apriori algorithm [2] and employs a *breadth-first search* strategy. To address certain limitations of the Two-Phase algorithm, several alternatives based on the pattern growth paradigm [16] have been developed. The effectiveness of two-phase algorithms is often contingent on the number of generated candidates. Consequently, various methods have been devised to create tighter upper bounds and reduce candidate generation [3,39]. In pursuit of more efficient UIM, researchers introduced one-phase algorithms that bypass the generation of candidates. HUI-Miner [31] is the first to introduce this concept, and subsequent algorithms have predominantly focused on optimization. FHM [14], for example, introduces the concept of co-occurrence based pruning to reduce the search space and explores this space using a depth-first search strategy, resulting in a seven-fold speed improvement over HUI-Miner. EFIM [45], a pattern growth one-phase algorithm, is developed to address certain limitations of *utility-list* methods. It introduces novel upper bounds and incorporates techniques like database projection, ultimately achieving a speed improvement of two to three orders of magnitude compared to HUI-Miner. In summary, UIM research has concentrated on both reducing the number of candidates and optimizing computation for improved efficiency.

For high utility subgraph mining, Shinoda et al. [38] extend UIM by considering the external weighted frequency and introduce the GWF-mining and CWF-mining algorithms in complex graph data. Khare et al. [23] introduced high utility subgraph mining incorporating edge weights and developed a distributed solution on Apache Spark and MapReduce. UGMINE [4] incorporates a tighter upper bound, denoted as *RMU*, by exploiting the feature of the rightmost path extension in gSpan. All these algorithms focus on graph transaction databases. Our method aims to perform the mining task in single graph databases.

3. Basic concepts

This section reviews some basic graph theory concepts necessary for describing our method.



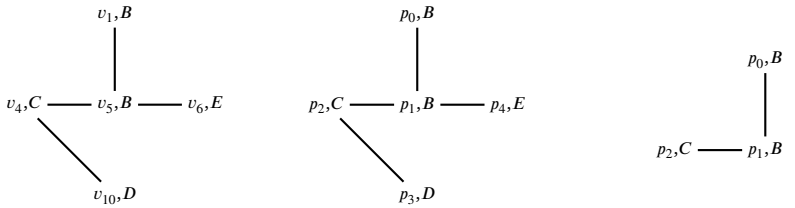
(a) Data graph G.

Table of external utilities

A	6
B	1
C	1
D	4
E	4
F	5

(b) Labels' external utilities.

Fig. 2. Single graph database G associated with external utilities of vertex labels.



(a) S, $S \subseteq G$. (b) P. (c) P', $P' \subseteq P$.

Fig. 3. Example subgraphs.

Definition 1 *(Graph).* Given a label set Σ, an (undirected) *graph* $G = (V_G, E_G, \lambda_G)$ consists of a vertex set V_G, an edge set $E_G \subseteq V_G \times V_G := \{(v_1, v_2) \mid v_1, v_2 \in V_G, v_1 \neq v_2\}$, and a labeling function $\lambda_G : V_G \cup E_G \to \Sigma$ that maps each vertex and each edge of the graph to an element of Σ.

The graph database considered in this paper contains only one graph, which we refer to as a large graph. Fig. 2 shows a single graph database, used in all the examples in this paper. Each vertex has two elements: a vertex ID v_i and a vertex label in $\Sigma = \{A, B, C, \cdots, F\}$. For simplicity, in the examples of this paper, we consider that all edge labels are identical to a uniform label a, which is omitted in the figures.

Definition 2 *(Subgraph).* A graph $S = (V_S, E_S, \lambda_S)$ is a *subgraph* of another graph $G = (V_G, E_G, \lambda_G)$ (G is a supergraph of S) if $V_S \subseteq V_G$, $E_S \subseteq E_G$ and $\forall v \in V_S \cup E_S$, $\lambda_S(v) = \lambda_G(v)$, written as $S \subseteq G$.

For example, as shown in Fig. 3 (a), the graph S is a subgraph of the data graph G in Fig. 2 since $V_S \subseteq V_G$, $E_S \subseteq E_G$ and $\forall u \in V_S \cup E_S$, $\lambda_S(u) = \lambda_G(u)$.

Definition 3 *(Pattern).* A *pattern* $P = (V_P, E_P, L_P)$ is a graph used as a query against another graph.

For example, as illustrated in Fig. 3 (b), P is a pattern that can serve as a query for the data graph G in Fig. 2.

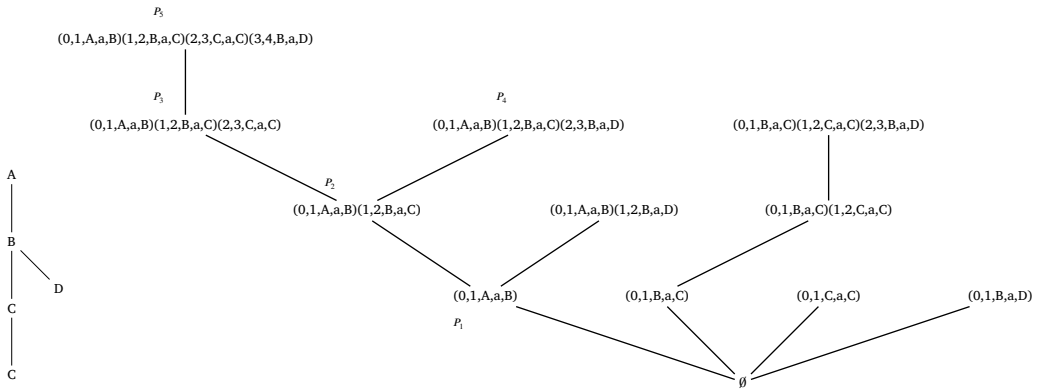


Fig. 4. Example graph and its minimum DFS code tree.

Definition 4 *(Isomorphism).* Given two graphs $G_1 = (V_{G_1}, E_{G_1}, \lambda_{G_1})$ and $G_2 = (V_{G_2}, E_{G_2}, \lambda_{G_2})$, G_1 is *isomorphic* to G_2 if there is an bijection $f : V_{G_1} \to V_{G_2}$ satisfying

a) $\forall v \in V_{G_1}$, $\lambda_{G_1}(v) = \lambda_{G_2}(f(v))$;

b) $\forall (v_1, v_2) \in E_{G_1}$, $s.t.$ $(f(v_1), f(v_2)) \in E_{G_2}$, $\lambda_{G_1}(v_1, v_2) = \lambda_{G_2}(f(v_1), f(v_2))$.

If G_1 is *isomorphic* to G_2, then it is denoted as $G_1 \simeq G_2$ and the mapping f is called an *isomorphism*.

For example, as shown in Fig. 3, there is a bijection between S and P, $f : V_S \to V_P$ such that $f(v_1) = p_0$, $f(v_4) = p_2$, $f(v_5) = p_1$, $f(v_6) = p_4$, $f(v_{10}) = p_3$. Therefore, $S \simeq P$ and the bijection f is an isomorphism.

Definition 5 *(Subgraph isomorphism).* Given a pattern $P = (V_P, E_P, \lambda_P)$ and a graph $G = (V_G, E_G, \lambda_G)$, if P is isomorphic to a subgraph of G, we say that P is *subgraph isomorphic* to G. An *occurrence* of P is an isomorphism from P to a subgraph of G.

For example, as mentioned above, S is a subgraph of G, and P is isomorphic to S. Therefore, P is subgraph isomorphic to G, and the bijection f is an occurrence of P in G.

4. High utility subgraph mining in a single graph

In this section, we first design the utility function for single graph databases. Then, we define the high utility subgraph mining problem and propose utility upper bounds for pruning the search space. Finally, we design the HUSM algorithm.

4.1. High utility subgraph mining in a single graph

In our method, each pattern is represented as a minimum DFS code, an edge sequence based on a specific order [12,42]. The minimum DFS code represents a pattern uniquely, and it is a canonical label of the pattern. For example, the pattern P shown in Fig. 3 (b) is uniquely represented by its minimum DFS code $(0, 1, B, a, B)$, $(1, 2, B, a, C)$, $(2, 3, C, a, D)$, $(1, 4, B, a, E)$ and this code is a canonical label of P. It is crucial to check that a DFS code is minimum before applying pruning because other DFS codes are not useful. Mining subgraphs in a single graph is equivalent to mining their corresponding canonical labels.

We employ the minimum DFS code tree as the enumeration tree $\mathcal{T}$, where each node corresponds to the minimum DFS code of a pattern. Given a minimum DFS code $\alpha = (e_0, e_1, \cdots, e_m)$, any minimum DFS code $\beta = (e_0, e_1, \cdots, e_m, e_{m+1})$ is referred to as a child of α, while α is considered the parent of β in $\mathcal{T}$. We present the following lemma according to the extension rules of $\mathcal{T}$.

Lemma 1. *For an enumeration tree, all nodes on a branch originating from a 1-edge pattern adopt the minimum DFS code of the 1-edge as their common prefix.*

For example, the left side of Fig. 4 depicts a graph, and the right side illustrates the enumeration tree when mining subgraphs in that graph. P_1 is the minimum DFS code of a *1-edge* pattern, and all its descendants lie on a branch rooted at P_1. It can be observed that the minimum DFS codes of all the descendants share P_1 as a common prefix. For convenience in subsequent references, we designate the vertex whose ID is 0 in P_1 as the **initial vertex**, denoted as I_0.

Definition 6 *(Maximum extension radius constraint (MER)).* Given a data graph G and a user-specified threshold *maximum extension radius constraint* (*MER*). Let P be a pattern, I_0 be the initial vertex of P, and f be an occurrence of P to G. The occurrence f is said to satisfy the *MER* if $\max_{v \in f(V_P)} d(f(I_0), v) \leq MER$ where $d(f(I_0), v)$ is the distance (length of the shortest path) between the vertices $f(I_0)$ and v in G.

Table 1
Computation of *CMNI* of the subgraph P.

p	p_0,B	p_1,B	p_2,C	p_3,D	p_4,E
$f_i(p)$	v_1	v_5	v_4	v_0×	v_6
	v_3	v_8	v_9	v_{10}	v_7
	v_{19}	v_{13}	v_{12}	v_{11}	v_{14}
	v_{21}	v_{16}	v_{17}		v_{15}

Table 2
CMNI table of the subgraph P'.

s	s_0,B	s_1,B	s_2,C
$f_i(s)$	v_1	v_5	v_4
	v_3	v_8	v_9
	v_{19}	v_{13}	v_{12}
	v_{21}	v_{16}	v_{17}

Our algorithm considers occurrences that satisfy Definition 6 as valid occurrences. For example, as illustrated in Fig. 3, with the initial vertex of P being p_0, the isomorphism from P to S can be regarded as an occurrence f in G. If we set *MER* to 2, then f is considered valid. However, if we replace $f(p_3) = v_{10}$ with $f(p_3) = v_0$ to obtain f', f' is not valid because the distance from v_0 to $f(I_0) = v_1$ is 3, which exceeds the specified *MER* of 2.

Intuitively, FSM utilizes the count of occurrences as support measure for mining frequent subgraphs. However, the count of occurrences lacks the *downward closure property*. Subsequently, in later studies, several support metrics such as *maximum independent sets (MIS)* [26], *harmful overlap (HO)* [13], and *minimum image based support (MNI)* [7] are introduced to calculate occurrences based on vertex overlap. They satisfy the *downward closure property*.

Since using the MIS and HO measures transforms the FSM problem in NP-complete problems, the most commonly employed metric is *MNI*, which can be computed in linear time. We adopt the concept of *MNI* to our context and define the *constrained minimum image based support*.

Definition 7 *(Constrained minimum image based support (CMNI))*. Given a data graph G, a pattern P, let $f(P,G) = \{f_1, f_2, \cdots, f_n\}$ be the set of occurrences of P in G that *satisfy MER*. The *constrained minimum image based support (CMNI)* of P is $CMNI(P) = \min_{v \in V_P} |\{f_i(v) : i = 1,2,\cdots,n\}|$.

For example, consider the pattern P shown in Fig. 3 (b). We obtain five occurrences $\{f_1, \cdots, f_5\}$ in G shown in Fig. 2. The computation of *CMNI* of P in G is shown in Table 1. The vertices in the first row of the table are arranged in the order of their appearance in the minimum DFS code of P, and each vertex p_j $(j = 0, \cdots, 4)$ corresponds to a column containing $\{f_i(p_j) | i = 1, \cdots, 5\}$. Since all the occurrences need to be valid, we should remove the vertex v_0 in the column associated with p_3 because its corresponding occurrence does not satisfy the *MER*. The final table obtained is called the ***CMNI* table**. Then according to Definition 7, $CMNI(P) = 2$.

Lemma 2. *The constrained minimum image based support (CMNI) satisfies the downward closure property.*

Proof. The *downward closure property* of the *CMNI* follows from that of *MNI* [12]. □

For example, consider the subgraph $P' \subseteq P$ in Fig. 3, its *CMNI* table is shown in Table 2, and we can get its $CMNI(P') = 4 > CMNI(P)$. This is because the invalid vertices in the domains of the *CMNI* table in a subgraph remain invalid in the *CMNI* table of its subsequently extended larger subgraphs. Therefore, the *CMNI* of a graph will never be greater than the *CMNI* of its subgraphs, thus satisfying the *downward closure property*.

In our specific problem, we consider the data graph G in a single graph database. When dealing with a pattern P in G, we can associate an inner utility to P, its *CMNI*, which depends on the graph structures of P and G. In real-world applications, distinct vertex labels often represent different values. Therefore, we need to deal with data graphs whose vertex labels are assigned external utilities. For example, the table in Fig. 2 (b) shows the vertex labels' external utilities in G. We can consider the average external utility of the vertex labels in P as the external utility of P. Now, we introduce the utility of a subgraph as follows.

Definition 8 *(Utility of a pattern)*. Given a pattern P and a data graph G, the *subgraph utility* of P in G is $u(P) = CMNI(\mathrm{P}) \times (\sum_{v \in V_P} u(\lambda(v))/|V_P|)$ where $u(\lambda(v))$ is the external utility of $\lambda_P(v)$.

For example, consider the pattern P and the data graph G again. According to the labels' external utilities shown in Fig. 2 (b), we obtain $\sum_{v \in V_P} u(\lambda(v))/|V_P| = 11/5 = 2.2$. Then, following Definition 8, $u(P) = 2 \times 2.2 = 4.4$.

Definition 9 *(High utility subgraph (HUS))*. Given a data graph G, two user-specified thresholds *minUtil* and *MER*. A pattern $P = (V_P, E_P, \lambda_P)$ is called a *high utility subgraph* if its utility satisfies $u(P) \geq minUtil$.

For example, consider *minUtil*= 4 and *MER*= 2. Given that the utility of P is $u(P) = 4.4$, it can be concluded that the pattern P depicted in Fig. 2 qualifies as a *high utility subgraph*.

Problem statement. Given a data graph G, two user-specified thresholds *minUtil* and *MER*. The *high utility subgraph mining problem* is to discover all the *HUS* in G.

4.2. Constraint satisfaction problem model

To construct the *CMNI* table for each subgraph, we solve the constraint satisfaction problem (*CSP*) in GraMi [12]. The *CSP* model in our problem consists of three components: $(\mathcal{X}, \mathcal{D}, C)$, where (i) $\mathcal{X}$ is a set of vertices; (ii) $\mathcal{D}$ denotes a set of domains $D(x_v)$ for each vertex x_v in $\mathcal{X}$; and (iii) C contains a set of constraints. Each solution of the *CSP* corresponds to a valid occurrence of pattern P, which satisfies the *MER*.

Definition 10 *(Constraint satisfaction problem (CSP)).* Given a data graph $G = (V_G, E_G, \lambda_G)$ and a pattern $P = (V_P, E_P, \lambda_P)$, the constraint satisfaction problem (*CSP*) for P in G is $(\mathcal{X}, \mathcal{D}, C)$ where:

(i) For each $v \in V_P$, there exists a variable $x_v \in \mathcal{X}$;

(ii) For each $x_v \in \mathcal{X}$, there is a $D(x_v) \subseteq V_G$;

(iii) C contains a series of constraints: (a) $\forall x_1, x_2 \in \mathcal{X},\ x_1 \neq x_2$; (b) $\forall v \in V_P,\ \lambda_G(x_v) = \lambda_P(v)$; (c) $\forall (v_1, v_2) \in E_P,\ \lambda_G(x_{v_1}, x_{v_2}) = \lambda_P(v_1, v_2)$; (d) $\max\limits_{v \in V_P} d(x_{I_0}, x_v) \leq MER$.

For example, consider the data graph G in Fig. 2 (a) and the pattern P in Fig. 3 (b). The *CSP* for P in G is defined with, $\mathcal{X} = (x_{s_0}, x_{s_1}, x_{s_2}, x_{s_3}, x_{s_4})$, $\mathcal{D} = \{(v_0, \cdots, v_{21}), (v_0, \cdots, v_{21}), (v_0, \cdots, v_{21}), (v_0, \cdots, v_{21}), (v_0, \cdots, v_{21})\}$, and C ($MER = 2$). Then, we solve the *CSP* and record each solution in a *CMNI* table. For example, $(x_{s_0}, x_{s_1}, x_{s_2}, x_{s_3}, x_{s_4}) = (v_1, v_5, v_4, v_{10}, v_6)$ is one of the solutions for P in G. We place these five vertices in their respective columns in the table. By repeating this process, we can obtain the final *CMNI* table for P as shown in Table 1. Then, we can compute the *CMNI*(P) from the table.

4.3. Pruning strategies

The utility of a pattern does not satisfy the *downward closure property*. For example, consider the subgraph $P' \subseteq P$ in Fig. 3 (c), its *CMNI* table is shown in Table 2 and its $CMNI(P') = 4$. According to the labels' external utilities in Fig. 2 (b), we obtain $\sum_{v \in V_{P'}} u(\lambda(v))/|V_{P'}| = 1$. Then by Definition 8, $u(P') = 4 \times 1 = 4$. If we set $minUtil = 4.2$, then P is a *HUS* while P' is not.

Hence, we cannot directly prune the enumeration tree with the utility value. We need to define a utility upper bound that satisfies the *downward closure property* to identify candidates and thus achieve pruning of the enumeration tree. Next, based on Definition 6, we shall define three utility upper bounds, called *IVU*, *EU*, and *CEU*. We begin by defining the vertex set corresponding to each $f(I_0)$ and subsequently employ it to establish the subsequent utility upper bounds.

Definition 11 *(Extension space vertices (ESV)).* Given a data graph G and a pattern P, let $f(P, G) = \{f_1, f_2, \cdots, f_n\}$ be the set of occurrences of the subgraph P in G that *satisfy MER*. The *extension space vertices* associated with f_i are defined as: $ESV(P, f_i) = \{v \in V_G \mid d(f_i(I_0), v) \leq MER\}$. The *extension space vertices* of P are defined as: $ESV(P) = \{\ ESV(P, f_i) | i = 1, \cdots, n\}$.

For example, consider the pattern P in Fig. 3 (b), which has five occurrences in the data graph G. As mentioned above, the initial vertex of P is the vertex p_0. Let $MER = 2$. We observe that there are only four distinct $f(I_0)$ in the first column of P's *CMNI* table. It is evident that among the five occurrences, two $f(I_0)$ share the vertex v_1, one $f(I_0)$ occupies the vertex v_3, one $f(I_0)$ occupies the vertex v_{19}, and another occupies the vertex v_{21}. Consequently, we have only four distinct *ESV*, each encompassed by one of the four dashed lines labeled as *Area 1, Area 2, Area 3*, and *Area 4* in Fig. 2.

Next, we utilize Definition 11 to define a basic utility upper bound.

Definition 12 *(Initial vertex utility (IVU)).* Given a data graph G and a pattern P. P's corresponding *ESV* set is $ESV(P) = \{ESV_1, ESV_2, \cdots, ESV_m\}$. The *initial vertex utility* is: $IVU(P) = \sum\limits_{i=1}^{m} \max\limits_{v \in ESV_i} u(\lambda_P(v))$.

Since for any occurrence f, $f(V_P) \subseteq \bigcup\limits_{i=1}^{m} ESV_i$, then *IVU* is an utility upper bound of $u(P)$. For example, we aim to determine the maximum utility within the vertices in *Area 1, Area 2, Area 3*, and *Area 4* in Fig. 2. The maximum utility within these areas is $\{6, 6, 4, 4\}$ respectively. Then we obtain $IVU(P) = 6 + 6 + 4 + 4 = 20 > u(P) = 4.4$.

IVU serves as a basic utility upper bound, analogous to *TWU* and *GWU*. We introduce two tighter upper bounds based on *IVU*.

Definition 13 *(Extension space utility (EU)).* Given a data graph G, a pattern P, and the corresponding *ESV* set $ESV(P) = \{ESV_1, ESV_2, \cdots, ESV_m\}$, each $ESV_i \in ESV(P)$ has a maximum value $u_i = \max\limits_{v \in ESV_i} u(\lambda_P(v))$, the *extension space utility* $EU(P)$ is the sum of the top $CMNI(P)$ largest values in $\{u_1, \cdots, u_m\}$.

Since for any occurrence f, $f(V_P) \subseteq \bigcup_{i=1}^{m} ESV_i$, then $\max_{v \in ESV_i} u(\lambda_P(v)) \geq \sum_{v \in V_P} u(\lambda(v))/|V_P|$. Therefore we have $EU(P) \geq u(P)$. For example, as mentioned above, the maximum utility within each area is $\{6, 6, 4, 4\}$ respectively. Since $CMNI(P) = 2$, according to Definition 13, we have $EU(P) = 6 + 6 = 12 > u(P) = 4.4$.

Definition 14 *(CMNI extension space utility (CEU))*. Given a data graph G, a pattern P, and P's corresponding ESV set $ESV(P) = \{ESV_1, ESV_2, \cdots, ESV_m\}$, for each $ESV_i \in ESV(P)$, let $u_i = \max_{v \in ESV_i \wedge \lambda_P(v) \geq \lambda_P(I_0)} u(\lambda_P(v))$. We define the *CMNI extension space utility* as the sum of the top $CMNI(P)$ largest values in $\{u_1, u_2, \cdots, u_m\}$, denoted as $CEU(P)$.

During the extension process of a pattern P, the labels contained in P's descendants in the enumeration tree will never be greater than $\lambda_P(I_0)$ in lexicographic order. Therefore, $\max_{v \in ESV_i} u(\lambda_P(v)) \geq \max_{v \in ESV_i \wedge \lambda_P(v) \geq \lambda_P(I_0)} u(\lambda_P(v)) \geq \sum_{v \in V_P} u(\lambda(v))/|V_P|$. So $CEU(P)$ is a utility upper bound. For example, the label A is less than $\lambda_P(I_0) = B$, so we do not consider the external utility of label A in each *Area*. Then, the maximum external utility of each area is $\{4, 5, 4, 4\}$. Since $CMNI(P) = 2$ and the top 2 largest external utilities are $\{5, 4\}$, then we have $u(P) < CEU(P) = 5 + 4 = 9 < EU(P)$.

Lemma 3. *The three utility upper bounds IVU, EU, and CEU satisfy the downward closure property.*

Proof. Given a pattern P and any of its descendants P_d, where P is a subgraph of P_d, the *downward closure property* means $IVU(P_d) \leq IVU(P)$, $EU(P_d) \leq EU(P)$, and $CEU(P_d) \leq CEU(P)$. Let $C_1(P_d)$ and $C_1(P)$ represent the first columns of the *CMNI* tables for P_d and P respectively. Because P is a subgraph of P_d, we have $C_1(P_d) \subseteq C_1(P)$, which results that $ESV(P_d) \subseteq ESV(P)$. Therefore, $\sum_{ESV_i \in ESV(P_d)} \max_{v \in ESV_i} u(\lambda_{P_d}(v)) \leq \sum_{ESV_i \in ESV(P)} \max_{v \in ESV_i} u(\lambda_P(v))$, which implies that $IVU(P_d) \leq IVU(P)$. Since $ESV(P_d) \subseteq ESV(P)$ and $CMNI(P_d) \leq CMNI(P)$ (Lemma 2), we obtain $EU(P_d) \leq EU(P)$. Similarly, we can prove that $CEU(P_d) \leq CEU(P)$. □

For example, consider the subgraph B-E (denoted as *subP*) as a subgraph of the pattern P in Fig. 3, and it has six occurrences in the data graph G when $MER = 2$. According to Definition 11, *subP* has six elements in $ESV(subP)$. Then based on Definition 12, we have $IVU(subP) = 6 + 6 + 6 + 4 + 4 + 4 = 30$, which is greater than $IVU(P)$. The *CMNI* value of *subP* is 4. According to Definition 13, $EU(subP) = 6 + 6 + 6 + 4 = 22$, which is greater than $EU(P)$. According to Definition 14, the set of maximum values for each *ESV* is $\{4, 4, 5, 4, 4, 4\}$, and $CEU = 5 + 4 + 4 + 4 = 17$, which is greater than $CEU(P)$.

4.4. HUSM algorithm

Our algorithm adheres to the framework of GraMi, comprising two core components. The initial component revolves around the subgraph search process, while the subsequent component is dedicated to computing subgraph utility and its corresponding upper bound. In Algorithm 1, we present our approach, denoted as HUSM. The subgraph extension and search space pruning process are delineated in Algorithm 2, and the computation of utility and utility upper bound is elaborated in Algorithm 3.

Algorithm 1: HUSM.

```
   Input: G: data graph, minUtil and MER: user-specified settings;
   Output: Result: all the high utility subgraphs in G;
1  Result ← ∅;
2  Let 1-edges be the set of all the edges in G whose CEU are not less than minUtil;
3  for each 1-edge ∈ 1-edges do
4      if u(1-edge) ≥ minUtil then
5          Result ← Result ∪ 1-edge;
6      end
7      Result ← Result ∪ SubgraphExtension (1-edge, G, minUtil, MER, 1-edges);
8  end
9  return Result;
```

Algorithm 1 is the framework of our HUSM method. It can discover all high utility subgraphs in a data graph G with two user-specified settings: *minUtil* and *MER*. The algorithm begins by initializing an empty set, *Result*, to record the high utility subgraphs (Line 1). Subsequently, it collects all edges in graph G whose *CEU* are not less than *minUtil* (Line 2). This step serves as an initial pruning to avoid the generation of numerous unnecessary subgraphs in the subsequent extension process. Then, the algorithm iterates over each *1-edge* ∈ *1-edges* (Line 3). It calculates the subgraph utility for each *1-edge* and checks if the utility value is greater than or equal to the specified threshold *minUtil* (Line 4). If its utility value satisfies *minUtil*, the *1-edge* is recorded as a high utility subgraph in *Result* (Line 5). Regardless of whether it is a high utility subgraph or not, after the determination, the algorithm calls Algorithm 2 for this *1-edge* to perform further expansion (Line 7). Any resulting high utility subgraphs are then recorded in *Result*. Finally, the algorithm returns *Result* as the output, containing all the high utility subgraphs in the data graph G (Line 9).

Algorithm 2: SubgraphExtension.

Input: *k-edge*: currently growing subgraph with size k, G: data graph, *minUtil* and *MER*: two user-specified settings, *1-edges*: all the *1-edge*;
Output: *Result*: all the high utility subgraphs in G;
1 *extensions* ← ∅;
2 **for** *each 1-edge* ∈ *1-edges and* $v \in$ *R-path*$_{k\text{-}edge}$ **do**
3 **if** *1-edge can be used to extend* v **then**
4 Let $(k+1)$-*edge* be the extension of *k-edge* with the *1-edge*;
5 **if** $(k+1)$-*edge is not already generated* **then**
6 *extensions* ← *extensions* ∪ $(k+1)$-*edge*;
7 **end**
8 **end**
9 **end**
10 **for** *each* $(k+1)$-*edge* ∈ *extensions* **do**
11 (*CEU*($(k+1)$-*edge*), u($(k+1)$-*edge*)) ← ValueCompute($(k+1)$-*edge*, *MER*, G);
12 **if** u($(k+1)$-*edge*) ≥ *minUtil* **then**
13 *Result* ← *Result* ∪ c;
14 **end**
15 **if** *CEU*($(k+1)$-*edge*) ≥ *minUtil* **then**
16 SubgraphExtension($(k+1)$-*edge*, G, *minUtil*, *MER*, *1-edges*);
17 **end**
18 **end**
19 **return** *Result*;

Algorithm 2 is designed to extend subgraphs and efficiently prune the search space. Like GraMi, the process is divided into two distinct phases: The first stage is the subgraph extension phase. The algorithm begins by initializing an empty set called *extensions* to record all newly generated $(k+1)$-*edges* from *k-edge* (Line 1). The algorithm iterates through the set *1-edges* and examines each vertex $v \in$ *R-path*$_{k\text{-}edge}$ (Line 2), where *R-path*$_{k\text{-}edge}$ represents the vertices on the rightmost path of the *k-edge*. If this *1-edge* can be used to extend the vertex v, then it adds the *1-edge* to the subgraph *k-edge* to generate $(k+1)$-*edge* (Lines 3-4). If the newly generated $(k+1)$-*edge* has not been generated before, it is recorded in the *extensions* set (Lines 5-7). The other stage is the search space pruning phase. The algorithm iterates over all newly generated $(k+1)$-*edge* in the set *extensions* (Line 10). For each $(k+1)$-*edge*, Algorithm 3 is called to calculate its utility upper bound (*CEU*) and utility value (Line 11). If the utility value of the $(k+1)$-*edge* satisfies the user-specified *minUtil*, the algorithm includes it as a high utility subgraph in *Result* (Lines 12-14). If the utility upper bound (*CEU*) of the $(k+1)$-*edge* satisfies *minUtil*, the algorithm continues to call Algorithm 2 for further extension of the $(k+1)$-*edge* (Lines 15-17). Otherwise, no extension is carried out, achieving the goal of pruning the search space. Finally, the algorithm returns the set *Result* (Line 19).

Algorithm 3: ValueCompute.

Input: *k-edge*: a subgraph of size k, *MER*: the user-specified setting, G: a data graph;
Output: *CEU*: the utility upper bound; the utility of *k-edge* in G;
1 *UtilityList* ← ∅, *CEUList* ← ∅;
2 Construct *CMNI* table for *k-edge*;
3 **for** *each* $v \in V_{k\text{-}edge}$ **do**
4 Add to $u(\lambda_{k\text{-}edge}(v))$ to *UtilityList*;
5 **end**
6 u(*k-edge*) ← *CMNI* × $\sum${*UtilityList*}/|*UtilityList*|;
7 **for** *each* $f(I_0)$ *of the first column in CMNI table* **do**
8 Construct corresponding *ESV*;
9 *MaxValue* ← 0;
10 **for** *each* $v \in$ *ESV* **do**
11 **if** $\lambda_{k\text{-}edge}(v) \geq \lambda_{k\text{-}edge}(I_0)$ *and* $u(v) >$ *MaxValue* **then**
12 *MaxValue* ← $u(v)$;
13 **end**
14 **end**
15 Add *MaxValue* to *CEUList*;
16 **end**
17 Sort the list *CEUList* in descending order;
18 *CEU*(*k-edge*) ← $\sum_{i=1}^{CMNI}$ *CEUList.get*(i);
19 **return** *CEU*($(k+1)$-*edge*), u($(k+1)$-*edge*);

Algorithm 3 can compute the utility upper bound and utility value for a given subgraph. The algorithm takes as input the given subgraph *k-edge*, the user-specified setting *MER*, and the data graph G. The algorithm first initializes two lists, *UtilityList* and *CEUList* (Line 1). *UtilityList* records the utility values of all vertices in the *k-edge* and is used to calculate the utility value of the *k-edge*. *CEUList* records the maximum utility value among all vertices in each constructed *ESV*, which is used for the subsequent calculation of the utility upper bound. Then, the algorithm constructs the *CMNI* table for the *k-edge* (Line 2), ensuring that each occurrence satisfies the

Algorithm 4: GetBHUSFromProcess.

```
   Input: A growing subgraph c, the user-specified threshold minUtil, the set BHUS to record all the BHUS;
   Output: All the BHUS;
 1 Extensions ← all potential children by adding an edge to c;
 2 if u(c) ≥ minUtil and ∀ ex ∈ Extensions, ex is not high utility subgraph then
 3 |   BHUS ← BHUS ∪ {c};     // c is a MaxBHUS;
 4 end
 5 if u(c) < minUtil then
 6 |   for each ex ∈ Extensions do
 7 |   |   if u(ex) ≥ minUtil then
 8 |   |   |   BHUS ← BHUS ∪ ex;     // ex is a MinBHUS;
 9 |   |   end
10 |   end
11 end
12 return BHUS;
```

user-specified setting *MER*. Next, the algorithm enters the specific calculation process. First, the algorithm uses *UtilityList* to record the utility values of all vertices in the *k-edge* (Lines 3-5). Then, according to Definition 8, it calculates the utility value of the *k-edge* and assigns it to u(*k-edge*) (Line 6). Second, the algorithm computes the utility upper bound (*CEU*) for the *k-edge*. It starts by iterating through the first column of the *CMNI* table, where each element represents a $f(I_0)$ (Line 7). Then, the algorithm constructs the corresponding *ESV* for each $f(I_0)$ based on Dijkstra's algorithm (Line 8). It iterates through all the vertices in the *ESV* (Line 10). The vertices used in the computation need to satisfy a condition that the labels of the vertices are lexicographically not less than the label of I_0 (Line 11). During this process, the algorithm uses *MaxValue* to record the maximum utility value among all vertices that satisfy this condition in a *ESV* (Line 12) and records each *MaxValue* in *CEUList* (Line 15). After that, the algorithm sorts the elements in *CEUList* in descending order (Line 17), takes the top *CMNI* elements, sums them, and assigns the sum to *CEU(k-edge)* (Line 18). Finally, the algorithm returns the computed upper bound and utility values (Line 19).

5. Optimization: concise representation for high utility subgraphs

As the introduction states, the two classical concise representations [22]: closed frequent subgraphs and maximal frequent subgraphs cannot be applied in high utility subgraph mining. We introduce a novel concise representation based on minimum DFS codes, and it can reduce information overlap in the mining results generated by our HUSM algorithm.

Definition 15 *(Boundary high utility subgraph).* Let P be a high utility subgraph. P is called a *boundary high utility subgraph* (*BHUS*) if, in the minimum DFS code tree, its parent is not a high utility subgraph or all its children are not high utility subgraphs. A *BHUS* is considered a *maximal boundary high utility subgraph* (*MaxBHUS*) if it has no high utility subgraphs as children. It is considered a *minimal boundary high utility subgraph* (*MinBHUS*) if its parent is not a high utility subgraph. Notice that, in particular, we consider the root node (empty subgraph) as a high utility subgraph.

For example, consider the minimum DFS code tree as shown in Fig. 4, we suppose that P_1, P_2, P_3, P_4, and P_5 are high utility subgraphs. P_4 and P_5 have no children with high utility subgraphs. Therefore, by Definition 15, only P_4 and P_5 are *BHUS*. When all the nodes on the path of two *BHUS* are high utility subgraphs, we only need to save the two *BHUS*. Since each parent's minimum DFS code is a prefix of its children's minimum DFS code in the enumeration tree, we can obtain the other nodes by *BHUS*'s minimum DFS code. For example, P_1, P_2, and P_3 are on the path from $\emptyset$ to P_5, P_1 and P_2 are on the path from $\emptyset$ to P_4. By removing the last edge of the minimum DFS code of P_5, we can obtain P_3, P_2, and P_1. Similarly, through P_4, we can obtain P_2 and P_1. Therefore, we need only to save P_4 and P_5 instead of all the subgraphs $\{P_1, P_2, P_3, P_4, P_5\}$.

Algorithm 4 illustrates the process of obtaining *BHUS* during the high utility subgraph mining procedure. Before Algorithm 4 is invoked to collect *BHUS*, whether the current growing subgraph c and all of its children are high utility subgraphs is known. Therefore, based on Definition 15, if c is high utility and none of c's children are high utility subgraphs, then c is considered as *MaxBHUS* (Lines 2-4). If c is not high utility and there exists a high utility subgraph *ex* among c's children, then *ex* is considered as *MinBHUS* since c is *ex*'s parent and is not high utility (Lines 5-11).

Algorithm 5 demonstrates the process of obtaining all *BHUS* from the results of high utility subgraph mining, denoted as *Result*. Since all subgraphs in the mining results are high utility subgraphs, the algorithm needs to check whether the children or the parent of a given subgraph *k-edge* exist in *Result* according to Definition 15. Based on Lemma 1, to determine if at least one of the children of *k-edge* exists in *Result*, the algorithm searches for a subgraph in *Result* with size $k+1$ such that its minimum DFS code's prefix is the minimum DFS code of *k-edge* (Lines 3-6). To check if the parent of *k-edge* exists in *Result*, it searches for a subgraph in *Result* with size $k-1$ such that its minimum DFS code is the prefix of the minimum DFS code of *k-edge* (Lines 7-10).

Algorithm 6 shows the process of obtaining all high utility subgraphs from the *BHUS* set. If a *BHUS* is *MaxBHUS* and *MinBHUS* simultaneously (Lines 3-5), it is added to the set *HUS*. Each *MaxBHUS* has a corresponding *MinBHUS*. The *MaxBHUS* is the smallest descendant among the *MinBHUS*, while the *MinBHUS* is the largest ancestor in size among the *MaxBHUS*. If a *BHUS* belongs to *MaxBHUS* (but not to *MinBHUS*), then the algorithm iterates through all *MaxBHUS* and searches for their largest ancestors within all

Algorithm 5: GetBHUSFromResults.

```
   Input: All the high utility subgraphs Result;
   Output: All the BHUS;
 1 BHUS ← ∅;
 2 for k-edge ∈ Result do
 3     if ∄ (k+1)-edge ∈ Result, (k+1)-edge is a child of k-edge then
 4         BHUS ← BHUS ∪ k-edge;     // k-edge is a MaxBHUS;
 5         continue;
 6     end
 7     if ∄ (k−1)-edge ∈ Result, (k−1)-edge is not the parent of k-edge then
 8         BHUS ← BHUS ∪ k-edge;     // k-edge is a MinBHUS;
 9         continue;
10     end
11 end
12 return BHUS;
```

Algorithm 6: Recover.

```
   Input: BHUS, the set of all the boundary high utility subgraphs;
   Output: All high utility subgraphs HUS;
 1 HUS ← ∅;
 2 for each bhus ∈ BHUS do
 3     if bhus is both MaxBHUS and MinBHUS then
 4         HUS ← HUS ∪ bhus;
 5     end
 6     if bhus is MaxBHUS then
 7         for all MinBHUS ∈ BHUS do
 8             if MinBHUS is the maximum size ancestor of bhus then
 9                 HUS ← HUS ∪ all the nodes on the path between the MinBHUS and bhus;     // use minimum DFS code
10             end
11         end
12     end
13 end
14 return HUS;
```

Table 3
Datasets and properties.

Dataset name	Vertex number	Distinct vertex labels	Edge number	density
Citeseer	3,312	6	4,732	Medium
LastFM	7,624	18	27,806	Dense
Gnutella	8,846	22	31,839	Dense
Mico	100,000	29	1,080,298	Dense
Amazon	89,741	54	897,811	Dense
YouTube	43,154	18	647,447	Dense

MinBHUS. This process still utilizes the minimum DFS code. Once each pair is found, all nodes along the path in the minimum DFS code tree can be recovered (Lines 6-12).

6. Experiments

In this section, we present experimental evaluations to assess the performance of HUSM. For comparison, the baseline algorithm employs the upper bound *IVU*, which is adapted from the classical upper bounds *TWU* and *GWU*. We compare the baseline algorithm with the two proposed tighter upper bounds, *EU* and *CEU*. Using each upper bound, we measure and analyze the number of candidates, runtime, and memory usage. Additionally, we compare the number of our concise representations with the total number of high utility subgraphs to evaluate their effectiveness. The algorithm is implemented in Java with JDK 11. The computer hardware configuration includes an i7-12700 CPU and 16 GB of memory. We collect six real-world single graph datasets, and their features are summarized in Table 3.

Citeseer: This dataset is acquired from GraMi [12]. It is commonly used in algorithm experiments for single graph database mining. Each vertex represents a publication labeled according to a computer science field, and each edge signifies a citation between publications.

LastFM: This dataset is collected from the website SNAP[1] and is related to the LastFM Asia Social Network, a popular social network for music enthusiasts. Vertices in this single graph dataset represent LastFM users from Asian countries, with labels extracted based on the artists liked by the users. Edges denote mutual follower relationships between users.

Gnutella[1]: This dataset represents an internet peer-to-peer network and is a snapshot of the Gnutella peer-to-peer file-sharing network from August 5th, 2002. In this dataset, vertices represent hosts in the Gnutella network topology, and edges represent connections between these hosts. We added 22 distinct random vertex labels since the original dataset lacked vertex labels.

Mico [12]: This dataset simulates Microsoft co-authorship information, and we use the collaboration graph in computer science for experiments. Vertices represent authors, with labels indicating the fields of interest for each author. Edges represent collaborations between two authors, with labels indicating the number of papers co-authored.

Amazon [17,34]: This dataset is associated with Amazon, sourced from a database that records Amazon product reviews from May 1996 to July 2014. The source data contains information such as product ID, product rating, category, and related products. We conduct a random sampling of the purchase information and the rating data across various categories to construct a single graph dataset. In this dataset, each vertex represents a product, and the vertex labels denote the categories of the corresponding products. The edges indicate the relationship between two items. Specifically, if the two products are related, there is an edge between the two vertices. Additionally, we compute the average ratings from users' reviews and employ them as external utilities for the vertex labels.

YouTube [10]: This dataset comprises statistics for specific YouTube videos crawled from February 22nd, 2007. The source data includes information such as video ID, category, video ratings, and related videos. We collect partial information and organize it into a single graph dataset. In this dataset, each vertex represents a video, and the vertex label indicates the category of the corresponding video. If video A is related to video B, there is an edge between vertices A and B. Furthermore, the average rating of videos within the same category is utilized as the external utilities.

In our experiment, we consider all the data graphs as undirected graphs. Our primary goal is to discover all connected high utility subgraphs within these datasets. Since Citeseer, LastFM, Gnutella, and Mico have no external utility values for vertex labels in the original datasets, we establish a standardized approach by setting the mean μ to 3.0 and the standard deviation σ to 1.0 for vertex labels in each graph data.

We initially set the parameter *MER* to 1 and conducted experiments with five different *minUtil* for each dataset. Additionally, we keep *minUtil* fixed and perform a series of experiments with *MER* set to 1, 2, 3, and 4, respectively. If the runtime exceeds 172,800 seconds (48 hours) or the algorithm runs out of memory, we assume there are no experimental results under the current setting.

6.1. Effectiveness evaluation

We first investigate whether HUSM can mine high utility subgraphs that satisfy the specified settings. As shown in Fig. 5 (a1)-(f1), with an increase in *minUtil*, the number of generated *HUS* decreases. For example, in the bar chart in Fig. 5 (a1) for the Citeseer dataset, it is evident that with the increase in *minUtil*, the number of *HUS* corresponding to each threshold is reduced compared to the previous threshold. Furthermore, as depicted in Fig. 5 (a2)-(f2), the chosen value of *MER* does not have a significant impact on the number of *HUS*. For example, when considering the LastFM and YouTube, it is noticeable that the height of *HUS* for each *MER* in the bar chart is nearly the same. This is because *HUS* satisfying the user-defined *minUtil* are often not very large, and, therefore, we only need to set a relatively small *MER* to mine almost all of the *HUS*.

We utilize three different utility upper bounds, *IVU*, *EU*, and *CEU*, to prune the search space. To demonstrate the effectiveness of these three upper bounds, we keep track of the number of candidates under different parameters, i.e., the number of subgraphs satisfying the upper bound with *minUtil*. Since the upper bound is always greater than or equal to the subgraph utility, the number of candidates is always greater than the number of *HUS*. For example, in the bar chart in Fig. 5 (a1) for the Citeseer dataset, we can observe that the height of the bar corresponding to candidates is always higher than that of the bar corresponding to *HUS* regardless of which upper bound is employed. Furthermore, tighter upper bounds tend to yield fewer candidates. Since $IVU \geq EU \geq CEU$, using *IVU* for pruning generates the most candidates while using *CEU* produces the fewest candidates. For example, in the bar chart for the Gnutella and Amazon in Fig. 5 (c1) and (e1), we can find that the number of candidates corresponding to *IVU*, *EU*, and *CEU* decreases in that order. This also indicates that *CEU* as a tighter upper bound results in a smaller number of candidates to be processed, making it more efficient for discovering *HUS*.

When *MER* remains constant and *minUtil* decreases, the number of candidates corresponding to each upper bound tends to increase. This is because as the threshold becomes lower, the number of subgraphs satisfying the *minUtil* also increases. During the mining process, all generated subgraphs have the potential to become *HUS*, but the actual *HUS* are often quite rare. To ensure that all of them are generated, we need to consider all subgraphs that satisfy the upper bound. As *minUtil* decreases, the search space expands, and the gap between the number of candidates and the number of *HUS* often increases. For example, in the Gnutella dataset's bar chart (c1), the difference between the number of candidates for each upper bound and the number of *HUS* becomes noticeably larger as *minUtil* decreases. When *minUtil* is fixed and *MER* increases, the increase in the number of candidates is quite apparent. This is because *MER* represents the maximum radius for the extension in each subgraph. If *MER* is larger, the size of candidates will also increase, leading to a further expansion of the search space. Since subgraphs satisfying *minUtil* are often relatively small, the number of candidates not satisfying *minUtil* also increases, widening the gap even further. As shown in the bar chart (a2) for the Citeseer

[1] http://snap.stanford.edu/index.html.

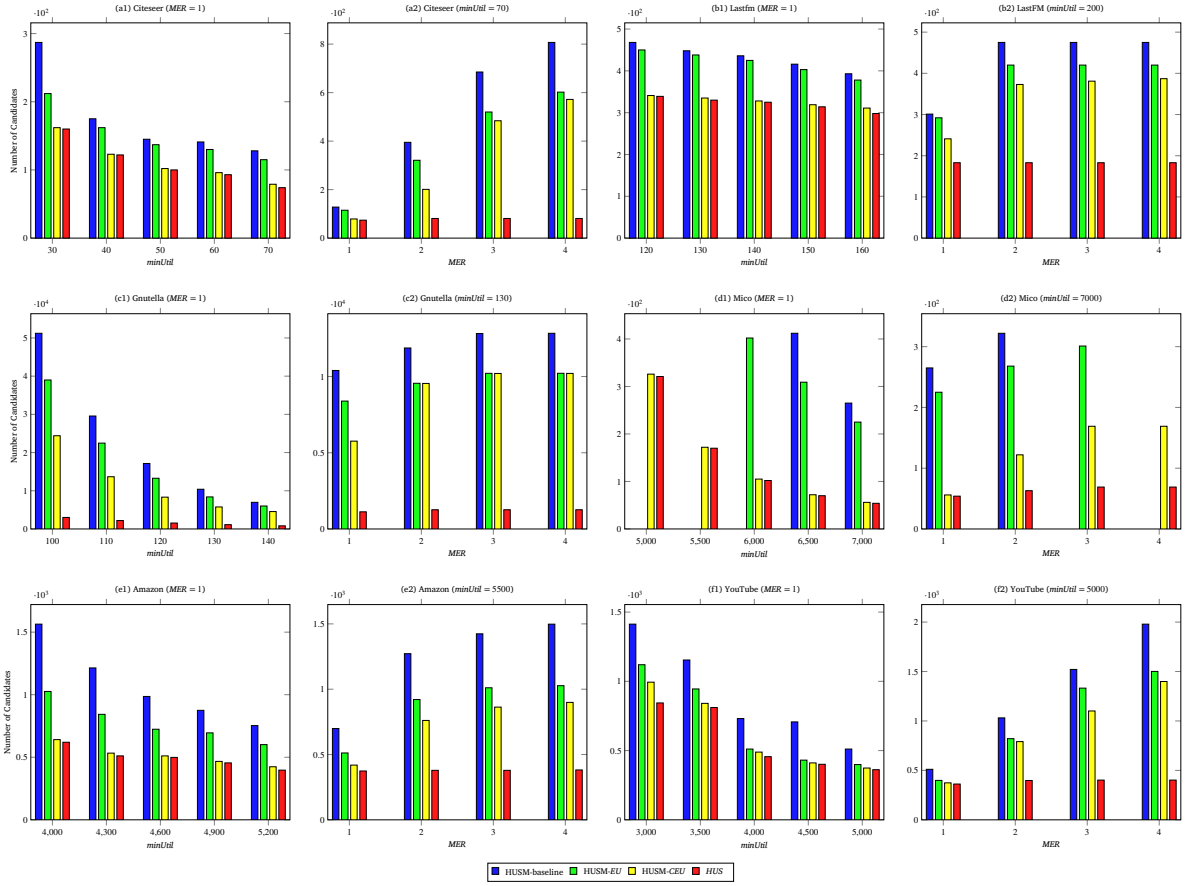


Fig. 5. Number of candidates and *HUS* for various *minUtil* and *MER*.

dataset, as *MER* increases, the number of candidates sharply rises, and the gap between candidates and *HUS* becomes increasingly significant. This further emphasizes that a smaller *MER* is often sufficient.

6.2. Runtime

This subsection compares the mining runtime required using each utility upper bound type. Fig. 6 (a1)-(f1) illustrates mining with *MER* fixed at 1 for each dataset using five different *minUtil*, while Fig. 6 (a2)-(f2) shows mining with *minUtil* fixed and *MER* set to 1, 2, 3, and 4, respectively.

When *MER* is fixed, the runtime of HUSM is related to the value of *minUtil*. A higher *minUtil* corresponds to a smaller search space, resulting in a shorter runtime. Similarly, when *minUtil* is fixed, a higher *MER* leads to a larger search space and, consequently, longer runtime. For example, the experimental results on the Citeseer dataset in Fig. 6 (a1) and (a2) show that the runtime curves exhibit monotonically decreasing and monotonically increasing trends, respectively. Furthermore, the use of different utility upper bounds results in different runtimes. Runtime is highly correlated with the number of candidates, as obtaining each candidate involves expensive subgraph isomorphism checks. Therefore, a larger number of candidates leads to a longer runtime. Given that the relationship between the number of candidates generated by the three upper bounds is $IVU \geq EU \geq CEU$, the runtime follows the same order. This relationship is particularly evident in the Gnutella dataset and shown in Fig. 6 (c1) and (c2). In addition, taking *CEU* as the utility upper bound allows for smaller values of *minUtil* and larger values of *MER*. As shown in Fig. 6 (d1), *IVU* can only have mining results when *minUtil* is set to 6500 and 7000, *EU* can only yield results when *minUtil* is set to 6000, 6500, and 7000. In contrast, *CEU* can obtain results for all five thresholds. Similarly, as depicted in Fig. 6 (d2), with a fixed *minUtil* of 7000, *CEU* can successfully perform mining tasks on larger *MER*. In summary, the tighter utility upper bound *CEU* proposed in our method reduces algorithm runtime by 20% or more and efficiently handles the high utility subgraph mining task in a larger search space.

The impact of *minUtil* and *MER* on the runtime differences between the three utility upper bounds is reflected in the search space. As mentioned earlier, as *minUtil* decreases, the algorithm's search space expands, and the difference in the number of candidates for the three upper bounds also increases. This, in turn, leads to a more significant difference in runtime. Conversely, as *minUtil* increases, the difference in runtime becomes smaller. For example, for the YouTube dataset, as shown in Fig. 6 (f1), the runtime differences between the three utility upper bounds increase as *minUtil* decreases and are at their maximum when $minUtil = 3000$ and at their minimum when $minUtil = 5000$. Similarly, as *MER* increases, the search space expands, and it has a similar impact on

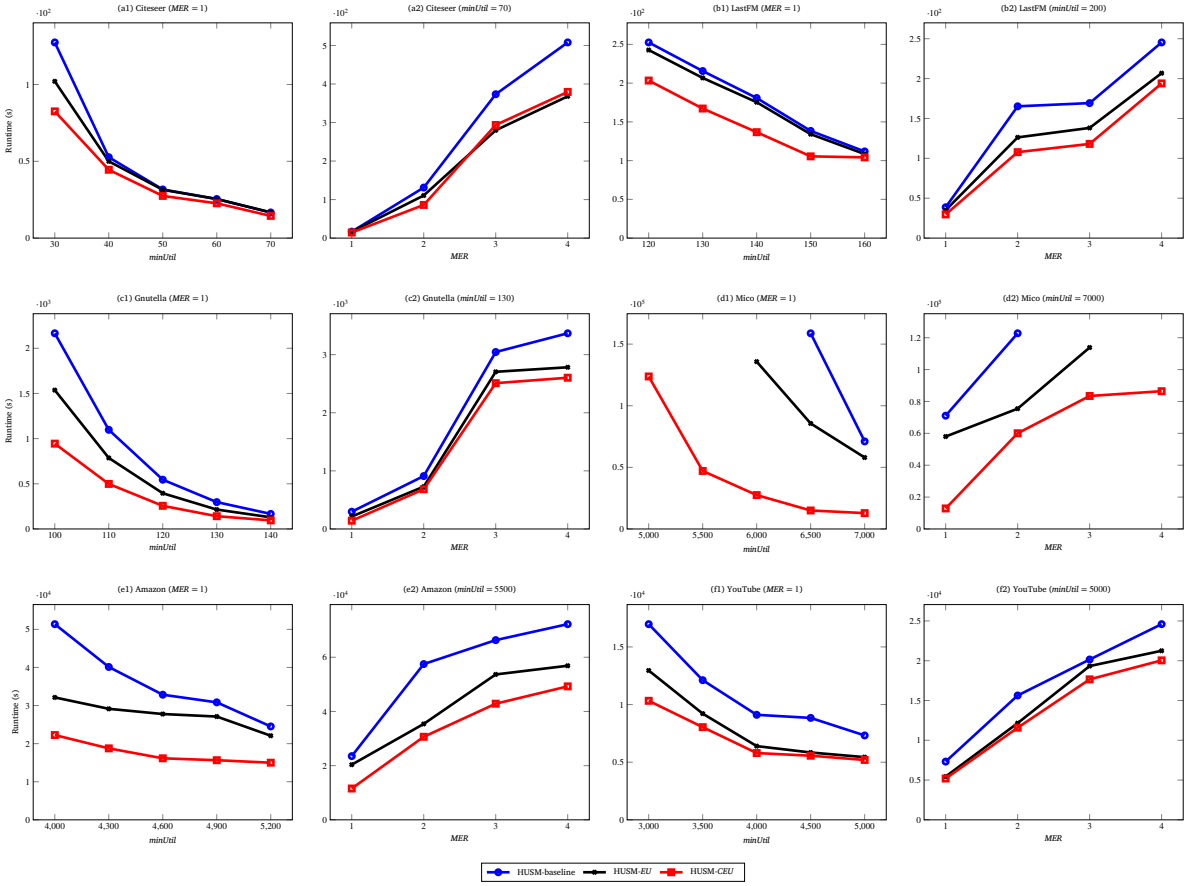


Fig. 6. Rumtime at various *minUtil* and *MER*.

the runtime differences between the three upper bounds. For example, in the Amazon dataset, as shown in Fig. 6 (e2), the runtime differences between the three utility upper bounds increase as *MER* increases.

6.3. Memory usage

In this part, we analyze the memory usage of the HUSM algorithm when using each of the three utility upper bounds. For each setting, we record the maximum memory usage of HUSM during its entire execution. Fig. 7 (a1)-(f1) displays the memory cost when *MER* is fixed at one and *minUtil* is varied across five values. Fig. 7 (a2)-(f2) shows the memory usage when *minUtil* is fixed and *MER* is set to 1, 2, 3, and 4.

The HUSM algorithm constructs the minimum DFS code tree using a depth-first search approach. When visiting a tree node, HUSM needs to store all nodes along the path from the root to the current node and their relevant information. Therefore, the maximum memory usage is highly correlated with the height of the enumeration tree. A larger number of candidates often results in a taller enumeration tree, so memory usage is related to the number of candidates. Since tighter utility upper bounds produce fewer candidates, based on the relationship of the three upper bounds, $IVU \geq EU \geq CEU$, *IVU* consumes more memory, while *CEU* consumes less. For example, for the Gnutella dataset, Fig. 7 (c1) and (c2) show that under each setting of the parameters *MER* and *minUtil*, the memory usage by HUSM follows the order $IVU \geq EU \geq CEU$. Moreover, the impact of the *minUtil* and *MER* parameters on memory usage is similar to their effect on the number of candidates. In general, *CEU*, as a tighter utility upper bound, effectively reduces the search space in high utility subgraph mining tasks, leading to reduced memory usage.

6.4. Concise representation

This subsection analyzes the effectiveness of the proposed concise representation by comparing the number of final *HUS* and *BHUS*. To maintain consistency, the experimental parameters are set the same as in the previous subsections. The experimental data obtained are recorded in Table 4. The upper part of the table shows the results when *MER* is fixed at 1, and five different *minUtil* values are tested for each dataset, showing the final number of *HUS* and *BHUS*. The lower part of the table shows the results when *minUtil* is fixed, and *MER* is set to 1, 2, 3, and 4, obtaining the final number of *HUS* and *BHUS* through experiments.

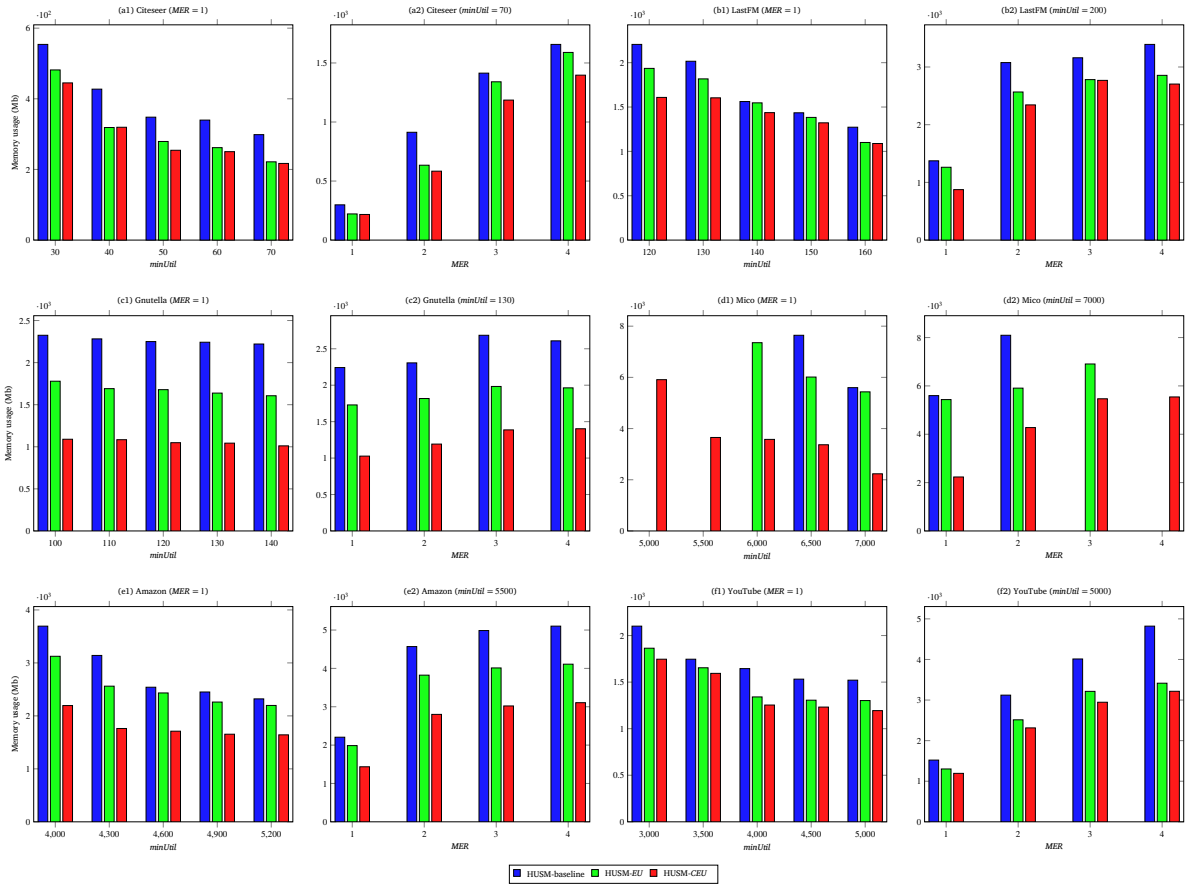


Fig. 7. Memory usage (Mb) and *HUS* at various *minUtil* and *MER*.

Table 4
Number of *HUS* and *BHUS* under various *minUtil* and *MER*.

Citeseer			**Gnutella**			**LastFM**		
minUtil	*HUS*	*BHUS*	*minUtil*	*HUS*	*BHUS*	*minUtil*	*HUS*	*BHUS*
30	160	160	120	339	339	100	3018	2677
40	122	122	130	330	330	110	2206	1967
50	100	100	140	325	325	120	1552	1372
60	93	93	150	314	314	130	1125	972
70	74	74	160	298	298	140	819	700
MER	*HUS*	*BHUS*	*MER*	*HUS*	*BHUS*	*MER*	*HUS*	*BHUS*
1	74	74	1	183	183	1	1125	972
2	81	75	2	183	183	2	1261	1091
3	81	75	3	183	183	3	1262	1092
4	81	75	4	183	183	4	1262	1092
Mico			**Amazon**			**Youtube**		
minUtil	*HUS*	*BHUS*	*minUtil*	*HUS*	*BHUS*	*minUtil*	*HUS*	*BHUS*
5000	321	119	120	620	467	100	844	411
5500	170	72	130	511	432	110	811	406
6000	102	53	140	499	431	120	456	274
6500	70	43	150	455	423	130	401	231
7000	54	36	160	397	397	140	362	224
MER	*HUS*	*BHUS*	*MER*	*HUS*	*BHUS*	*MER*	*HUS*	*BHUS*
1	54	36	1	374	374	1	362	224
2	63	42	2	379	376	2	398	236
3	69	48	3	379	376	3	402	239
4	69	48	4	382	377	4	402	239

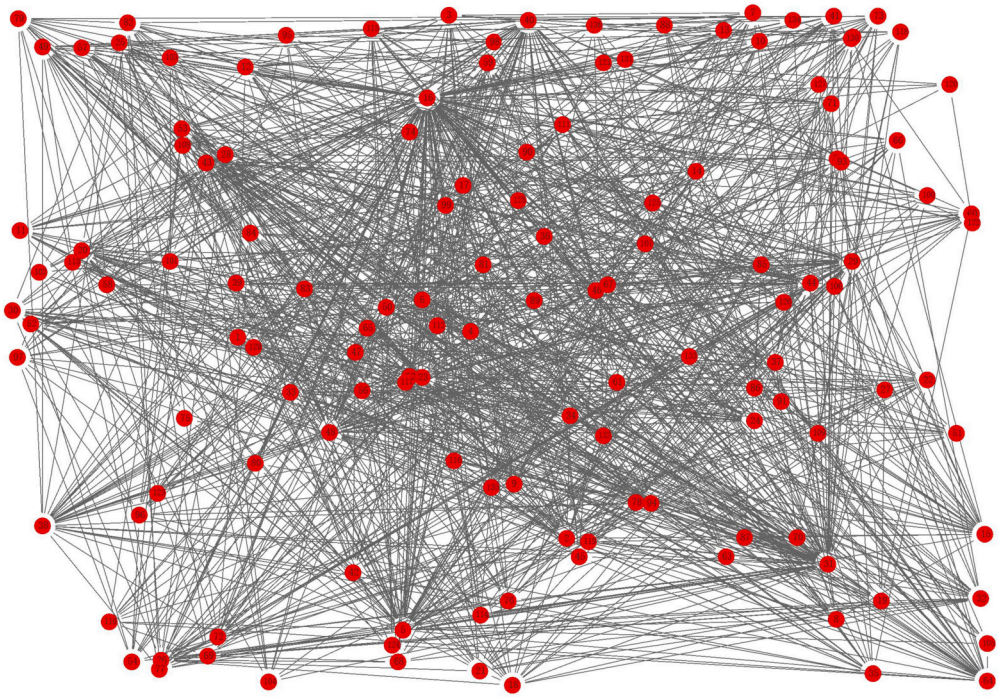


Fig. 8. Example of Amazon network.

Under each set of parameters, the number of *BHUS* is always less than or equal to the number of *HUS* because *BHUS* is a concise representation derived from *HUS*. Additionally, the number of *BHUS* is highly correlated with the number of *HUS*. Therefore, *BHUS* tends to decrease as *minUtil* decreases and increase as *MER* increases. For example, as shown in Table 4, for the Gnutella and LastFM datasets, we can observe that the changes in the number of *BHUS* closely follow the changes in the number of *HUS*.

As mentioned earlier, the reduction in the number of *HUS* achieved by our concise representation primarily results from the path between two *BHUS*. Therefore, the difference between *BHUS* and *HUS* counts mainly stems from the distribution of *HUS* in the enumeration tree. In the final results, the larger the maximum size of *HUS*, the better the conciseness effect of *BHUS*. This is because a larger *HUS* size implies that more *HUS* are mined, leading to a denser distribution. For example, in the Citeseer dataset, when *MER* is fixed at 1 for the five different *minUtil* settings, *BHUS* and *HUS* counts are equal. This is because the maximum size of the mined *HUS* is 1, making *BHUS* unnecessary for conciseness. However, when *MER* is fixed and increased, allowing larger subgraphs, *BHUS* counts decrease. In the case of the Lastfm and Mico datasets, *HUS*'s maximum size is greater than 1 for each parameter setting, making the conciseness effect of *BHUS* significant. In summary, when *minUtil* is smaller, or *MER* is larger, indicating a more extensive and larger set of mined *HUS*, *BHUS* can provide substantial conciseness. This underscores the effectiveness of the proposed concise representation.

6.5. Case study

To illustrate the effectiveness of our algorithm, we extract a smaller dataset from Amazon. This dataset consists of 136 vertices and 1220 edges. Each vertex represents a product, and for simplicity, we use integers from 0 to 135 to denote the original product IDs. The label of each product vertex corresponds to its product category. An edge exists between two products if they are related. Fig. 8 displays the network structure to represent this dataset visually.

Task: We set a threshold τ and independently mine frequent subgraphs and high utility subgraphs below this threshold. The results of the frequent subgraphs and high utility subgraphs are then compared to demonstrate that our HUSM algorithm can discover more valuable subgraphs, revealing more latent patterns.

Settings: We conduct statistical analysis for each product category. To enhance the contrast, we assign external utilities to each category inversely proportional to the count. In other words, the more frequent a category appears, the smaller its external utility. Table 5 details each product category's count and external utility.

Result 1: The threshold is $\tau = 4$. The final number of frequent subgraphs is 113, and the number of high utility subgraphs is 137. The following examples illustrate two different types of subgraphs. As shown in Fig. 9, (a) and (b) represent subgraphs that are frequent but not high utility, while (c) and (d) represent subgraphs that are not frequent but high utility. The frequent subgraphs primarily include labels such as Digital Music and Video Games. This is because, as indicated in Table 5, Digital Music and Video Games have the highest count. Therefore, these two product categories are more likely to form frequent subgraphs. However, the external utility values for these two categories are relatively low, indicating a perceived lower value for subgraphs containing these

Table 5
Properties of the labels.

Label	Count	External utility
Digital Music	26	0.3
Video Games	17	0.6
Pet Supplies	16	0.9
Clothing	15	1.2
Sports and Outdoors	10	1.5
Toys and Games	10	1.8
Kindle Store	8	2.1
Cell Phones and Accessories	8	2.4
Beauty	6	2.7
Office Products	6	3
Automotive	3	3.3
Baby	3	3.6
Books	2	3.9
Tools and Home Improvement	2	4.2
Musical Instruments	2	4.5
Amazon Instant Video	2	4.8

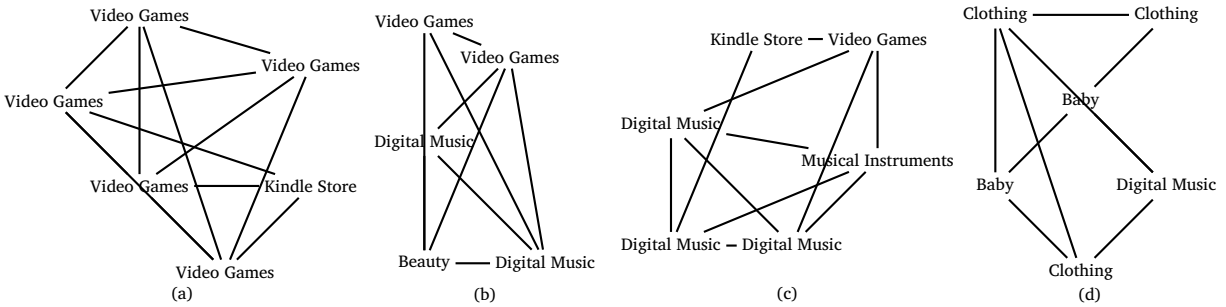


Fig. 9. Examples of frequent subgraphs and high utility subgraphs.

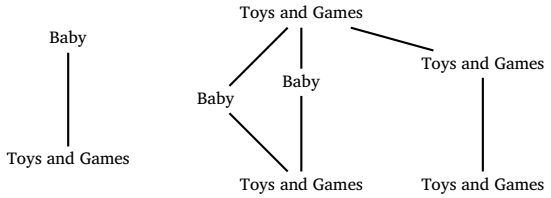


Fig. 10. Examples of the *BHUS*.

categories; the mined results of frequent subgraphs may also include subgraphs that are not high utility, such as (a) and (b). On the other hand, the high utility subgraphs mined by the HUSM algorithm, like (c) and (d), include the category Musical Instruments and Baby. As shown in Table 5, the count of the two categories is less than 4. Therefore, the two categories would not be mined as labels of a frequent subgraph. However, the HUSM algorithm successfully mines such subgraphs, uncovering valuable patterns that might be overlooked in frequent subgraph mining frameworks.

Result 2: Additionally, the algorithm obtains concise representations of the high utility subgraphs, ultimately reducing the 137 high utility subgraphs to 63. As shown in Fig. 10, we display two of these *BHUS*, covering the information of the original six high utility subgraphs. The algorithm only needs to store these two subgraphs to encompass the information of the six subgraphs mined initially. It demonstrates that our algorithm is capable of producing more concise mining results.

In summary, our designed algorithm HUSM, by integrating external utility and support, can provide more precise value assessments for subgraphs, facilitating the discovery of more valuable patterns. Additionally, it effectively minimizes information overlap in the mining results, resulting in more concise and meaningful results.

7. Conclusion and future work

This paper presents a utility function that combines *CMNI* with external utility. We propose three utility upper bounds, *IVU*, *EU*, and *CEU*, to efficiently prune the search space. Subsequently, we introduce the HUSM algorithm for mining high utility subgraphs in single graph databases. We conduct experiments using six real datasets: Citeseer, LastFM, Gnutella, Mico, Amazon, and YouTube. The experimental results demonstrate the effectiveness of our algorithm. Furthermore, we comprehensively compare the three upper

bounds based on three key aspects: the number of candidates, algorithm runtime, and memory usage. Our findings confirm that *CEU* serves as a tighter utility upper bound compared to *EU* and *IVU*, thereby enhancing the algorithm's efficiency. Additionally, we introduce a novel concise representation for high utility subgraphs based on minimum DFS code. Experimental results illustrate that our concise representation, denoted as *BHUS*, effectively reduces information overlap and identifies more valuable subgraphs. Our future work will focus on developing subgraph mining algorithms that leverage the special structure of subgraphs and do not require utility upper bounds for certain utility functions. These algorithms will be designed to apply to dynamic databases. This research direction has the potential to develop the high utility subgraph mining field further and make it more relevant in dynamic and evolving data environments.

CRediT authorship contribution statement

Zhaoming Chen: Writing – original draft, Software, Methodology, Conceptualization. **Cheng He:** Software, Data curation, Writing – review & editing. **Guoting Chen:** Supervision, Validation, Writing – review & editing. **Wensheng Gan:** Validation, Writing – review & editing. **Philippe Fournier-Viger:** Validation, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The data used are publicly available on internet.

Acknowledgement

This work has been partly supported by the Shenzhen Science and Technology Program (Basic Research Project, No. JCYJ20210324133003011), National Natural Science Foundation of China (No. 62272196 and 12171491), Guangzhou Basic and Applied Basic Research Foundation (Excellent Doctoral "Renewal", No. 2024A04J9971).

References

[1] E. Abdelhamid, I. Abdelaziz, P. Kalnis, Z. Khayyat, F. Jamour, ScaleMine: scalable parallel frequent subgraph mining in a single large graph, in: Proc. 16th IEEE Intern. Conf. High Performance Computing, Networking, Storage and Analysis, 2016, pp. 716–727.
[2] R. Agrawal, R. Srikant, Fast algorithms for mining association rules in large databases, in: Proc. 20th Intern. Conf. Very Large Data Bases, 1994, pp. 487–499.
[3] C.F. Ahmed, S.K. Tanbeer, B.S. Jeong, Y.K. Lee, Efficient tree structures for high utility pattern mining in incremental databases, IEEE Trans. Knowl. Data Eng. 21 (2009) 1708–1721.
[4] M.T. Alam, A. Roy, C.F. Ahmed, M.A. Islam, C.K. Leung, UGMINE: utility-based graph mining, Appl. Intell. 53 (2022) 49–68.
[5] N. Ashraf, R.R. Haque, M.A. Islam, C.F. Ahmed, C.K.S. Leung, J.J. Mai, B.H. Wodi, WeFreS: weighted frequent subgraph mining in a single large graph, in: Proc. 19th Industrial Conf. Data Mining, 2019, pp. 201–215.
[6] S. Bashir, A.R. Baig, HybridMiner: mining maximal frequent itemsets using hybrid database representation approach, in: Proc. 9th Intern. Pakistan Section Multitopic Conference, 2005, pp. 1–7.
[7] B. Bringmann, S. Nijssen, What is frequent in a single graph?, in: Proc. 12th Pacific-Asia Conf. Advances in Knowledge Discovery and Data Mining, 2008, pp. 858–863.
[8] X. Chen, J. Cai, G. Chen, W. Gan, A. Broustet, FCSG-Miner: frequent closed subgraph mining in multi-graph dataset, Instr. Sci. 665 (120363) (2024) 1–19.
[9] Z. Chen, X. Chen, G. Chen, W. Gan, Frequent subgraph mining in dynamic databases, in: Proc. 11th IEEE Intern. Conf. Big Data, 2023, pp. 5733–5742.
[10] X. Cheng, C. Dale, J. Liu, Statistics and social network of youtube videos, in: Proc. 16th Intern. Workshop on Quality of Service, 2008, pp. 229–238.
[11] F. Eichinger, M. Huber, K. Böhm, On the usefulness of weight-based constraints in frequent subgraph mining, in: Proc. 30th SGAI Intern. Conf. Innovative Techniques and Applications of Artificial Intelligence, 2011, pp. 65–78.
[12] M. Elseidy, E. Abdelhamid, S. Skiadopoulos, P. Kalnis, GraMi: frequent subgraph and pattern mining in a single large graph, Proc. VLDB Endow. 7 (2014) 517–528.
[13] M. Fiedler, C. Borgelt, Subgraph support in a single large graph, in: Proc. 7th IEEE Intern. Conf. Data Mining Workshops, 2007, pp. 399–404.
[14] P. Fournier-Viger, C.W. Wu, S. Zida, V.S. Tseng, FHM: faster high-utility itemset mining using estimated utility co-occurrence pruning, in: Proc. 21st Intern. Symp. Foundations of Intelligent Systems, 2014, pp. 83–92.
[15] W. Gan, J.C.W. Lin, H.C. Chao, P.S. Yu, Discovering high utility episodes in sequences, IEEE Trans. Artif. Intell. 4 (2023) 473–486.
[16] J. Han, J. Pei, Y. Yin, Mining frequent patterns without candidate generation, in: Proc. 25th ACM Intern. Conf. Management of Data, 2000, pp. 1–12.
[17] R. He, J. McAuley, Ups and downs: modeling the visual evolution of fashion trends with one-class collaborative filtering, in: Proc. 25th Intern. Conf. World Wide Web, 2016, pp. 507–517.
[18] J. Huan, W. Wang, J. Prins, Efficient mining of frequent subgraphs in the presence of isomorphism, in: Proc. 3rd IEEE Intern. Conf. Data Mining, 2003, pp. 549–552.
[19] A. Inokuchi, T. Washio, H. Motoda, An apriori-based algorithm for mining frequent substructures from graph data, in: Proc. 4th European Conf. Principles of Data Mining and Knowledge Discovery, 2000, pp. 13–23.
[20] M.A. Islam, C.F. Ahmed, C.K. Leung, C.S.H. Hoi, WFSM-MaxPWS: an efficient approach for mining weighted frequent subgraphs from edge-weighted graph databases, in: Proc. 22nd Pacific-Asia Conf. Knowledge Discovery and Data Mining, 2018, pp. 664–676.
[21] C. Jiang, F. Coenen, M. Zito, Finding frequent subgraphs in longitudinal social network data using a weighted graph mining approach, in: Proc. 6th Intern. Conf. Advanced Data Mining and Applications, 2010, pp. 405–416.
[22] C. Jiang, F. Coenen, M. Zito, A survey of frequent subgraph mining algorithms, Knowl. Eng. Rev. 28 (2013) 75–105.

[23] A. Khare, V. Goyal, S. Baride, S.K. Prasad, M. McDermott, D. Shah, Distributed algorithm for high-utility subgraph pattern mining over big data platforms, in: Proc. 24th IEEE Intern. Conf. High Performance Computing, 2017, pp. 263–272.
[24] M. Kuramochi, G. Karypis, Frequent subgraph discovery, in: Proc. 1st IEEE Intern. Conf. Data Mining, 2001, pp. 313–320.
[25] M. Kuramochi, G. Karypis, GREW - a scalable frequent subgraph discovery algorithm, in: Proc. 4th IEEE Intern. Conf. Data Mining, 2004, pp. 439–442.
[26] M. Kuramochi, G. Karypis, Finding frequent patterns in a large sparse graph, Data Min. Knowl. Discov. 11 (2005) 243–271.
[27] N. Le, B. Vo, L.B.Q. Nguyen, B. Le, OWGraMi: efficient method for mining weighted subgraphs in a single graph, Expert Syst. Appl. 204 (117625) (2022) 1–13.
[28] N. Le, B. Vo, U. Yun, B. Le, A novel approach to discover frequent weighted subgraphs using the average measure, Appl. Intell. 53 (2023) 19491–19504.
[29] N.T. Le, B. Vo, L.B. Nguyen, H. Fujita, B. Le, Mining weighted subgraphs in a single large graph, Inf. Sci. 514 (2020) 149–165.
[30] G. Lee, U. Yun, Mining weighted frequent sub-graphs with weight and support affinities, in: Proc. 12th Intern. Conf. Multi-Disciplinary Trends in Artificial Intelligence Workshop, 2012, pp. 224–235.
[31] M. Liu, J. Qu, Mining high utility itemsets without candidate generation, in: Proc. 21st ACM Intern. Conf. Information and Knowledge Management, 2012, pp. 55–64.
[32] Y. Liu, W. Liao, A. Choudhary, A two-phase algorithm for fast discovery of high utility itemsets, in: Proc. 9th Pacific-Asia Conf. Advances in Knowledge Discovery and Data Mining, 2005, pp. 689–695.
[33] C. Lucchese, S. Orlando, R. Perego, Fast and memory efficient mining of frequent closed itemsets, IEEE Trans. Knowl. Data Eng. 18 (2006) 21–36.
[34] J. McAuley, C. Targett, Q. Shi, A. van den Hengel, Image-based recommendations on styles and substitutes, in: Proc. 38th Intern. ACM SIGIR Conf. Research and Development in Information Retrieval, 2015, pp. 43–52.
[35] L.B. Nguyen, B. Vo, N.T. Le, V. Snasel, I. Zelinka, Fast and scalable algorithms for mining subgraphs in a single large graph, Eng. Appl. Artif. Intell. 90 (103539) (2020) 1–12.
[36] G. Preti, M. Lissandrini, D. Mottin, Y. Velegrakis, Mining patterns in graphs with multiple weights, Distrib. Parallel Databases 39 (2021) 281–319.
[37] F. Qiao, X. Zhang, P. Li, Z. Ding, S. Jia, H. Wang, A parallel approach for frequent subgraph mining in a single large graph using spark, Appl. Sci. 8 (230) (2018) 1–18.
[38] M. Shinoda, T. Ozaki, T. Ohkawa, Weighted frequent subgraph mining in weighted graph databases, in: Proc. 9th IEEE Intern. Conf. Data Mining Workshops, 2009, pp. 58–63.
[39] V.S. Tseng, B.E. Shie, C.W. Wu, P.S. Yu, Efficient algorithms for mining high utility itemsets from transactional databases, IEEE Trans. Knowl. Data Eng. 25 (2013) 1772–1786.
[40] N. Vanetik, E. Gudes, S. Shimony, Computing frequent graph patterns from semistructured data, in: Proc. 2nd IEEE Intern. Conf. Data Mining, 2002, pp. 458–465.
[41] C.W. Wu, Y.F. Lin, P.S. Yu, V.S. Tseng, Mining high utility episodes in complex event sequences, in: Proc. 19th ACM SIGKDD Intern. Conf. Knowledge Discovery and Data Mining, 2013, pp. 536–544.
[42] X. Yan, J. Han, gSpan: graph-based substructure pattern mining, in: Proc. 2nd IEEE Intern. Conf. Data Mining, 2002, pp. 721–724.
[43] J. Yang, W. Su, S. Li, M.M. Dalkiliç, WIGM: discovery of subgraph patterns in a large weighted graph, in: Proc. 12th SIAM Intern. Conf. Data Mining, 2012, pp. 1083–1094.
[44] X. Zhang, F. Lai, G. Chen, W. Gan, Mining high-utility sequences with positive and negative values, Inf. Sci. 637 (118945) (2023) 1–15.
[45] S. Zida, P. Fournier-Viger, J.C.W. Lin, C.W. Wu, V.S. Tseng, EFIM: a fast and memory efficient algorithm for high-utility itemset mining, Knowl. Inf. Syst. 51 (2017) 595–625.